# Algorítmica

## 2º Grado en Ingeniería Informática

# Guion de prácticas

## Algoritmos de exploración de grafos

## Sumario

Objetivo....2
Problemas....2
Tareas a realizar....4
Entrega de la práctica....4

## Índice de figuras

Figura 1: Representación del tablero....2
Figura 2: Tablero del solitario chino....2
Figure 3: Ciudades de España con distancias entre ellas....3

# Algoritmos de exploración de grafos

## 1. Objetivo

El objetivo de la práctica consiste en que el alumno sea capaz de analizar una serie de problemas y resolverlos mediante las técnicas de diseño de algoritmos basadas en exploración de grafos, como la vuelta atrás (Backtracking) o la ramificación y poda (Branch&Bound)

## 2. Problemas

Para cada uno de los problemas siguientes, se debe:

- Diseñar e implementar en C++ un algoritmo (Backtracking y/o Branch&bound) que resuelva el problema.
- Proponer un ejemplo de uso (instancia del problema) representativo, y explicar cómo funciona el algoritmo sobre este ejemplo, paso a paso.

### 2.1. Problema 1

En una liga universitaria de baloncesto participan $n$ equipos (suponemos que $n$ es par). En cada jornada se juegan $n/2$ partidos, que enfrentan a dos equipos, y dirigidos por un único árbitro. Existen $m$ árbitros disponibles, siendo $m > n/2$.
Cada equipo $i$ valora a cada árbitro $j$ con una puntuación $p(i, j)$ entre 1 y 10, indicando su preferencia por ese árbitro (un valor alto indica que le gusta el árbitro y un valor bajo que no le gusta). El objetivo es, dada una jornada concreta con unos emparejamientos entre equipos, asignar un árbitro distinto a cada partido, de manera que se maximice la puntuación total de los árbitros asignados, teniendo en cuenta las preferencias de todos los equipos. Esto se medirá mediante la suma de los productos de las puntuaciones asignadas por cada equipo al árbitro asignado a su propio partido.
Diseña e implementa un algoritmo de exploración de grafos para resolver el problema.

### 2.2. Problema 2

En el juego (solitario) del senku, también llamado solitario chino, se colocan 32 piezas iguales en un tablero de 33 casillas, tal y como se indica en las figura 1 y 2 (las “x" corresponden a posiciones no válidas):



*Figura 2: Tablero del solitario chino*

| | | | | | | |
|---|---|---|---|---|---|---|
| X | X | O | O | O | X | X |
| X | X | O | O | O | X | X |
| O | O | O | O | O | O | O |
| O | O | O | | O | O | O |
| O | O | O | O | O | O | O |
| X | X | O | O | O | X | X |
| X | X | O | O | O | X | X |

*Figura 1: Representación del tablero*

Solo se permiten movimientos de las piezas en vertical y horizontal. Una pieza solo puede moverse saltando sobre otra y situándose en la siguiente casilla, que debe estar vacía. La pieza sobre la que se salta se retira del tablero. Se consigue terminar con éxito el juego cuando queda una sola pieza en la

posición central del tablero (la que estaba inicialmente vacía). Diseña e implementa un algoritmo de exploración de grafos que encuentre una serie de movimientos para llegar con éxito al final del juego. En el siguiente video puedes encontrar a alguien ganando el juego: https://www.youtube.com/watch?v=1NEnM8FiQmk

## 2.3. Problema 3

Un distribuidor de cervezas Alhambra desea planificar una ruta de reparto eficiente desde su sede en Granada. El objetivo es visitar una sola vez cada una de las ciudades del mapa adjunto (ver figura 3) y regresar a Granada, minimizando el total de kilómetros recorridos. Para ello se podrá utilizar como entrada una matriz de adyacencia que representa las distancias entre las ciudades.

Se pide:

1. Diseñar e implementar un algoritmo de **backtracking** que:
   1. Genere todas las rutas posibles desde Granada.
   2. Cada ruta debe recorrer cada ciudad una única vez y regresar al punto de partida.
   3. Si hay varias rutas posibles, indicar la que tenga menor distancia total.
2. Diseñar e implementar un algoritmo de **branch and bound** que:
   1. Encuentre la ruta óptima (de menor distancia total) sin necesidad de generar todas las rutas posibles.
   2. Use una función de cota inferior para evitar explorar soluciones parciales peores que la mejor encontrada hasta el momento.

Para ambos apartados, debe utilizarse como entrada una matriz de distancias entre ciudades, como se muestra en el mapa. Granada será siempre la ciudad origen y final del recorrido.

Se valorará especialmente:

- Que el grafo de estados generado por ambos algoritmos sea finito y esté bien acotado, indicando las restricciones implícitas y explicitas que den lugar a ello.
- El uso correcto de la poda en la versión de branch and bound.
- Una explicación paso a paso del funcionamiento sobre una instancia pequeña (por ejemplo, 4 o 5 ciudades).

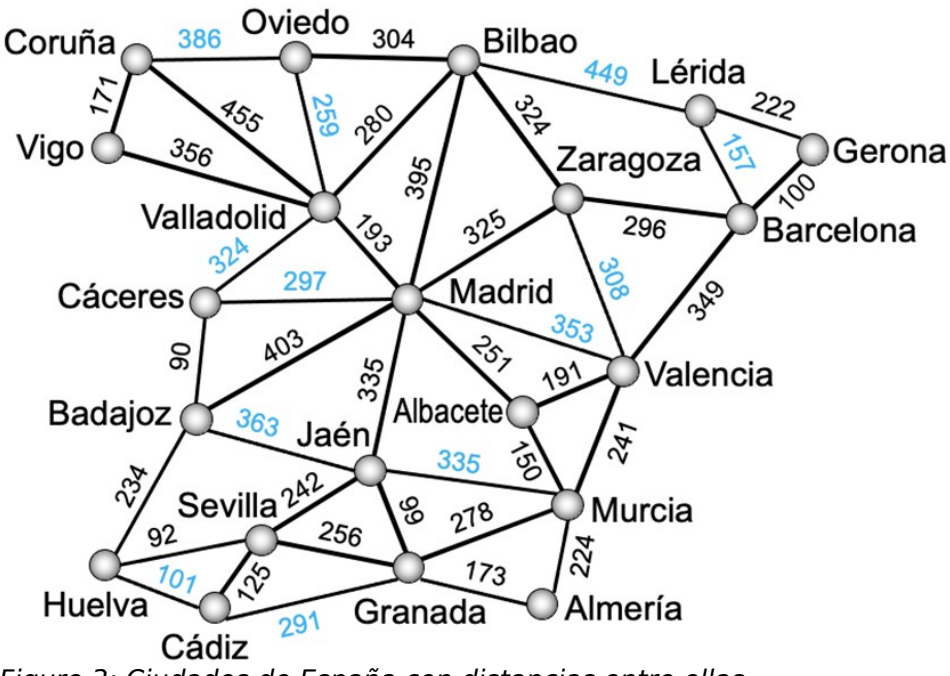



*Figure 3: Ciudades de España con distancias entre ellas*

## 3. Tareas a realizar

Se deben resolver todos los ejercicios propuestos en el apartado 2 de este guion. Cada ejercicio se valorará sobre 10 de la siguiente forma:

1. **(5 puntos)** **Diseño** del algoritmo
2. **(2 puntos)** **Ejemplo** paso a paso de la explicación del funcionamiento del algoritmo para una instancia pequeña propuesta por el grupo de estudiantes.
3. **(3 puntos)** Correcto funcionamiento de la **implementación**.

La valoración de la práctica se dará como una calificación numérica entre 0 y 10.

## 4. Entrega de la práctica

Se deberá entregar:

1. Un documento (memoria de prácticas) realizado en equipos, conteniendo la solución a los problemas propuestos, según los apartados descritos en el apartado 2 de este guion.
2. Los ficheros fuente con las diferentes implementaciones, junto con un makefile y un ejemplo de ejecución.

La práctica deberá ser entregada a través de PRADO, en la fecha y hora límite explicada en clase por el profesor. No se aceptarán, bajo ningún concepto, prácticas entregadas con posterioridad a la fecha límite indicada. La entrega de PRADO permanecerá abierta con, al menos, una semana de antelación antes de la fecha límite, por lo que todo alumno tendrá tiempo suficiente para entregarla.